CS 363M — Principles of Machine Learning
Fall 2026

Midterm exam · solutions

Name: _____ EID: _____

Course modality (circle one): In-person Online

Instructions

Read this page before you begin. Do not turn the page until you are told to start.

- There are **5 questions** worth a total of **100 points**. Point values are shown beside each question and part.
- Budget your time according to the point values. Computational parts generally take longer than conceptual parts.
- **Each question begins on its own page. Write your answer for each part in the blank space directly below it, and label it** with its part number — 1.1, 1.2, and so on.
- **You must submit your cheat sheet with your exam.** Write your name on it and hand it in stapled behind the last page. An exam handed in without its cheat sheet will not be graded until the sheet is produced.
- One double-sided cheat sheet is permitted.
- **Show your work on every computational part.** An answer with no work receives no credit. Correct work with an arithmetic slip receives most of the credit, so write down the steps even when you are unsure of the final number.
- Round to four decimal places unless a part tells you otherwise. Leaving an answer as an unreduced fraction is fine.
- A basic or scientific calculator is permitted. Phones, laptops, tablets and smart watches must be off and out of sight for the whole period.
- If a part seems ambiguous, write down the assumption you are making and answer under that assumption. A clearly stated assumption will be graded fairly.
- Keep your eyes on your own paper. Suspected academic dishonesty is referred to Student Conduct and Academic Integrity.
- If you finish early you may hand in your exam and leave quietly. In the last ten minutes, please stay seated so you do not disturb the others.

Good luck.

1. Evaluating classifiers (20 points)

A university wants to use an AI-text detector to check student essays. To test it, the detector is run on 400 essays whose authorship is known: 60 were written with AI and 340 by students. Of the 400 essays, the detector flags 50 as written with AI. Of these 50 flagged essays, 45 really were written with AI. Treat "written with AI" as the positive class.

1.1 (4 points) Write down the confusion matrix.

Solution

60 AI-written essays and 340 human-written essays.

	flagged	not flagged
written with AI	TP $= 45$	FN $= 15$
written by students	FP $= 5$	TN $= 335$

1.2 (2 points) What are precision and recall for this detector?

Solution

Precision $= 45/50 = 0.90$. Recall $= 45/60 = 0.75$.

1.3 (2 points) Do we care more about improving precision or recall for this problem, and why?

Solution

Precision. A false positive accuses an innocent student of cheating; a missed AI essay (false negative) is a smaller harm. A flag should almost always be right before anyone acts on it.

The detector also gives each essay a score between 0 and 1; a higher score means "more likely written with AI". Here are 8 essays, 4 written with AI and 4 by students:

essay	1	2	3	4	5	6	7	8
score	0.95	0.85	0.70	0.60	0.50	0.40	0.30	0.10
written with AI?	yes	yes	no	yes	no	yes	no	no

We say the detector flags an essay as AI-written if its score is over t.

1.4 (10 points) The ROC curve starts at $(\mathbf{FPR}, \mathbf{TPR}) = (0, 0)$, when t is above every score. Lower t past one essay at a time. What are the next three points on the curve?

Solution

As t passes each essay, from the highest score down, an AI essay moves the point up by $1/4 = 0.25$ and a human essay moves it right by 0.25. Essays 1 and 2 are AI-written and essay 3 is not, so the next three points are

$$(0,\ 0.25), \quad (0,\ 0.5), \quad (0.25,\ 0.5).$$

1.5 (2 points) Another detector has an AUC of 0.3 on the test essays. What does that tell you, and what simple change would improve it?

Solution

It is worse than random guessing (AUC 0.5): it tends to give human-written essays higher scores than AI-written ones. Flip it: use $1 -$ score (equivalently, flag the essays with low scores). That reverses the order of the essays, so the AUC becomes $1 - 0.3 = 0.7$.

2. Support vector machines (20 points)

2.1 (10 points) A soft-margin SVM returned $\boldsymbol{w} = (6, 8)$, $b = -20$.

point	C	D	E
x_1	1.5	1.25	3
x_2	1.5	1.5	0.5
label	$+$	$-$	$-$

(a) Compute $f(x)$ and the predicted class of each point.

(b) What is the width of the margin?

(c) Compute each point's distance to the decision boundary.

(d) Which points are on the margin? Which points are inside the margin? Which points are misclassified?

Solution

$f(\boldsymbol{x}) = 6x_1 + 8x_2 - 20$ and $\|\boldsymbol{w}\| = \sqrt{6^2 + 8^2} = 10$.

point	C	D	E
$f(\boldsymbol{x})$	1	-0.5	2
predicted class	$+$	$-$	$+$
distance $\lvert f(\boldsymbol{x})\rvert/\|\boldsymbol{w}\|$	0.1	0.05	0.2
$y\, f(\boldsymbol{x})$	1	0.5	-2

(b) Margin width $= 2/\|\boldsymbol{w}\| = 2/10 = 0.2$.

(d) On the margin: C ($y\, f = 1$). Inside the margin: D ($0 < y\, f = 0.5 < 1$), still correctly classified. Misclassified: E (label $-$, predicted $+$; $y\, f = -2$).

2.2 (4 points) The soft-margin SVM solves

$$\min_{\boldsymbol{w}, b, \boldsymbol{\xi}} \ \tfrac{1}{2}\|\boldsymbol{w}\|^2 + C\sum_i \xi_i \quad \text{subject to} \quad y_i(\boldsymbol{w}\cdot\boldsymbol{x}_i + b) \geq 1 - \xi_i, \ \ \xi_i \geq 0.$$

(a) What does the slack ξ_i measure, and why do we need it?

(b) Where does a training point lie, relative to the margin and the decision boundary, if $\xi_i > 1$?

Solution

(a) ξ_i measures how far point i falls short of being on its correct side, outside the margin. Without slack, the constraints $y_i(\boldsymbol{w}\cdot\boldsymbol{x}_i + b) \geq 1$ have no solution when the classes overlap; slack lets some points violate the margin, at a cost $C\xi_i$ each.

(b) On the wrong side of the decision boundary: misclassified. (At the solution $\xi_i = \max(0,\, 1 - y_i f(\boldsymbol{x}_i))$, so $\xi_i > 1$ means $y_i f(\boldsymbol{x}_i) < 0$.)

2.3 (4 points) Two soft-margin SVMs are trained on the same data, one with $C = 0.01$ and one with $C = 100$. For (a)–(c), circle one answer.

(a) Which has the wider margin?	(**$C = 0.01$**)	$C = 100$
(b) Which has more support vectors?	(**$C = 0.01$**)	$C = 100$
(c) Which is more likely to overfit?	$C = 0.01$	(**$C = 100$**)

(d) How would you choose C in practice, and why?

Solution

(a) $C = 0.01$: slack is cheap, so the SVM accepts many points inside the margin in exchange for a wider one. (b) $C = 0.01$: with a wider margin, more points fall on or inside it, and all of them are support vectors. (c) $C = 100$: violations are expensive, so the margin narrows to fit individual training points. (d) C is a hyperparameter: try several values and pick the one with the best accuracy on validation data (or by cross-validation). Training accuracy cannot choose it, because a larger C always fits the training data at least as well; only held-out data shows which C generalizes.

2.4 (2 points) You remove a training point that is not a support vector and retrain the SVM. What changes? Why?

Solution

Nothing. The solution depends only on the support vectors; a point outside the margin does not affect $\boldsymbol{w}$ or b.

3. Nonlinear regression (20 points)

3.1 (4 points) kNN regression is trained on six points:

x	1	2	4	5	7	8
y	3	6	4	8	10	12

Predict y at $x = 4.4$ using (a) $k = 1$ and (b) $k = 3$.

Solution

(a) The nearest point is $x = 4$, so the prediction is 4. (b) The three nearest points are $x = 4, 5, 2$ (distances $0.4, 0.6, 2.4$), so the prediction is $(4 + 8 + 6)/3 = 6$.

3.2 (6 points) For each of the following, name a hyperparameter that controls how closely the model fits the training data. Which setting of it makes overfitting more likely?

(a) Regression tree

Solution

The maximum depth: a larger depth (or a smaller minimum number of points needed to split).

(b) kNN regression

Solution

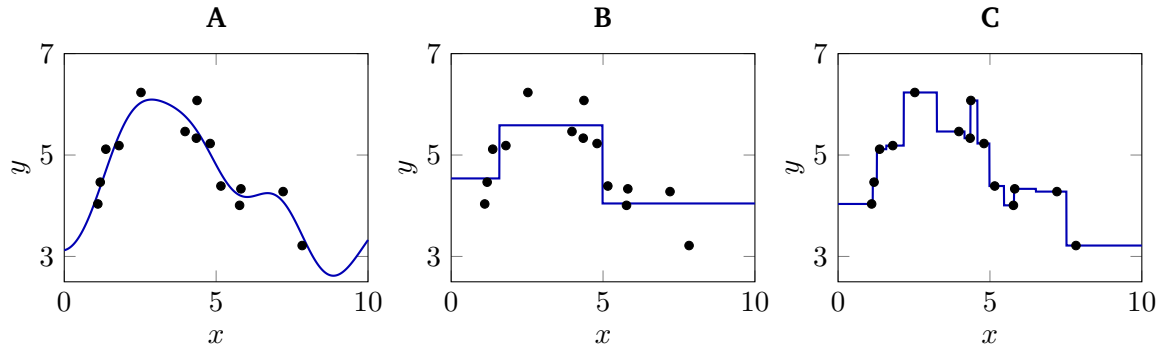
The number of neighbours k: a smaller k.

(c) SVR

Solution

C: a larger C (or a smaller ε).

3.3 (5 points) The figure shows three models fitted to the same data. Which panel shows a regression tree, which shows kNN with $k = 1$, and which shows SVR with a kernel? Explain each.



Solution

A is SVR with a kernel: a smooth curve. B is the regression tree: a few wide flat steps, one for each leaf, which predicts the average y of its points. C is kNN with $k = 1$: a narrow step for every training point, passing through every point.

3.4 (5 points) Suppose you use kNN to predict house prices from floor area (500 to 5,000 square feet) and number of bedrooms (1 to 6). Why might kNN do badly on these raw features? What would you do?

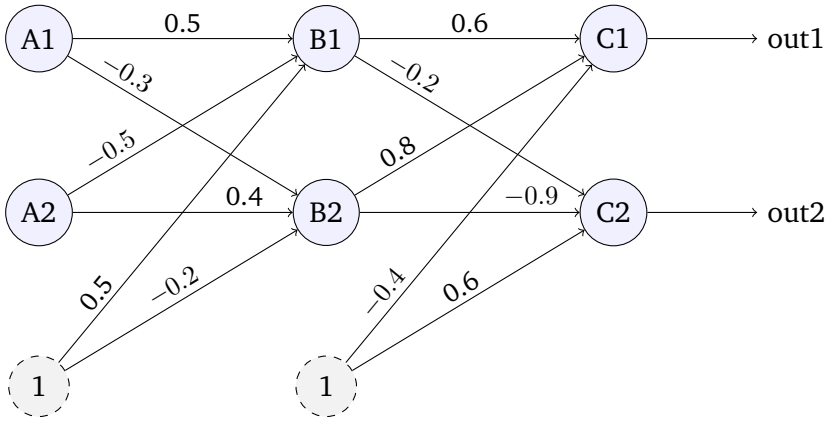
Solution

Distances are dominated by floor area: a difference of one bedroom counts as much as one square foot, so the nearest houses match on area and almost ignore bedrooms. Standardize each feature (subtract its mean, divide by its standard deviation) before computing distances.

4. Neural networks (30 points)

In this question, row j of a weight matrix $\boldsymbol{W}^{(l)}$ holds the weights into node j of layer l: $W^{(l)}_{ji}$ is the weight from node i of the previous layer to node j. The bias of node j is $b^{(l)}_j$, the weight on the input that is always 1.

4.1 (8 points) In the network below, out1 represents Class A and out2 represents Class B. The hidden nodes B1 and B2 use the ReLU activation, $\mathrm{ReLU}(z) = \max(0, z)$. The output nodes C1 and C2 have no activation: each outputs its weighted sum. The network predicts the class whose output is larger.



Show the forward pass, in matrix form, of the record with Feature1 $= 4$ and Feature2 $= 2$. How would this network classify the record (Class A or Class B)?

Solution

From the diagram:

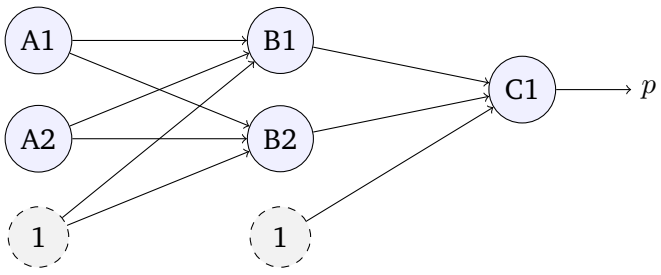
$$\boldsymbol{W}^{(1)} = \begin{pmatrix} 0.5 & -0.5 \\ -0.3 & 0.4 \end{pmatrix}, \quad \boldsymbol{b}^{(1)} = \begin{pmatrix} 0.5 \\ -0.2 \end{pmatrix}, \quad \boldsymbol{W}^{(2)} = \begin{pmatrix} 0.6 & 0.8 \\ -0.2 & -0.9 \end{pmatrix}, \quad \boldsymbol{b}^{(2)} = \begin{pmatrix} -0.4 \\ 0.6 \end{pmatrix}.$$

$$\boldsymbol{z}_B = \boldsymbol{W}^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)} = \begin{pmatrix} 0.5 \cdot 4 - 0.5 \cdot 2 + 0.5 \\ -0.3 \cdot 4 + 0.4 \cdot 2 - 0.2 \end{pmatrix} = \begin{pmatrix} 1.5 \\ -0.6 \end{pmatrix}, \quad \boldsymbol{B} = \mathrm{ReLU}(\boldsymbol{z}_B) = \begin{pmatrix} 1.5 \\ 0 \end{pmatrix}$$

$$\boldsymbol{C} = \boldsymbol{W}^{(2)}\boldsymbol{B} + \boldsymbol{b}^{(2)} = \begin{pmatrix} 0.6 \cdot 1.5 + 0.8 \cdot 0 - 0.4 \\ -0.2 \cdot 1.5 - 0.9 \cdot 0 + 0.6 \end{pmatrix} = \begin{pmatrix} 0.5 \\ 0.3 \end{pmatrix}$$

out1 $= 0.5 >$ out2 $= 0.3$: the network predicts Class A. (Forgetting the ReLU, and using $B_2 = -0.6$, gives out1 $= 0.02$ and out2 $= 0.84$, the wrong class.)

4.2 (14 points) The network below has one hidden layer and one output. A record with inputs $A_1 = 2$, $A_2 = 1$ and label $Y = 1$ is passed in.



	function	derivative
B layer: sigmoid	$f(z) = 1/(1+e^{-z})$	$f(z)\,(1-f(z))$
C layer: sigmoid	$p = C_1 = 1/(1+e^{-z_C})$	(see the loss)
loss: cross-entropy	$E = -Y \log p - (1-Y)\log(1-p)$	$\partial E/\partial z_C = p - Y$

Here z_C is C1's input sum, and p is the predicted probability that $Y = 1$. The weights during the forward propagation are

$$\boldsymbol{W}^{(1)} = \begin{pmatrix} 0.5 & -1.0 \\ 1.5 & 0.5 \end{pmatrix}, \quad \boldsymbol{b}^{(1)} = \begin{pmatrix} 0.5 \\ -1.0 \end{pmatrix}, \qquad \boldsymbol{W}^{(2)} = \begin{pmatrix} 1.0 & -2.0 \end{pmatrix}, \quad b^{(2)} = 0.5.$$

You do not need to forward propagate. The node output values during the forward propagation are

A layer	B layer	C layer
$A_1 = 2$	$B_1 = 0.62$	$C_1 = p = 0.33$
$A_2 = 1$	$B_2 = 0.92$	

Use a learning rate of 1. Compute every derivative with the weights above, the ones used in the forward propagation.

(a) Adjust $\boldsymbol{W_{11}^{(2)}}$, the weight from B1 to C1.

(b) Adjust $\boldsymbol{W_{11}^{(1)}}$, the weight from A1 to B1.

Solution

(a) The error at the output is $\partial E/\partial z_C = p - Y = 0.33 - 1 = -0.67$. Since $z_C = W_{11}^{(2)} B_1 + W_{12}^{(2)} B_2 + b^{(2)}$, $\partial z_C/\partial W_{11}^{(2)} = B_1 = 0.62$, so

$$\frac{\partial E}{\partial W_{11}^{(2)}} = -0.67 \cdot 0.62 = -0.415, \qquad W_{11}^{(2)} \leftarrow 1.0 - 1 \cdot (-0.415) = 1.415.$$

(b) The path is $W_{11}^{(1)} \to z_{B_1} \to B_1 \to z_C \to E$, where z_{B_1} is B1's input sum. Reuse -0.67 from (a), with the old weight $W_{11}^{(2)} = 1.0$:

$$\frac{\partial E}{\partial B_1} = -0.67 \cdot 1.0 = -0.67, \qquad \frac{\partial E}{\partial z_{B_1}} = -0.67 \cdot B_1(1 - B_1) = -0.67 \cdot 0.62 \cdot 0.38 = -0.158,$$

$$\frac{\partial E}{\partial W_{11}^{(1)}} = -0.158 \cdot A_1 = -0.158 \cdot 2 = -0.316, \qquad W_{11}^{(1)} \leftarrow 0.5 - 1 \cdot (-0.316) = 0.816.$$

(With unrounded node values the updates are 1.419 and 0.817; accept either.)

4.3 Short answers (8 points in all).

(a) (3 points) You train the same network twice with different learning rates and plot the training loss after each epoch. In run 1 the loss jumps up and down and ends higher than it started. In run 2 it goes down steadily, but very slowly. For each run, is the learning rate too large or too small? Why?

Solution

Run 1: too large. Each step overshoots the minimum, so the loss bounces around and can grow. Run 2: too small. Each step moves the weights only a little, so the loss falls slowly.

(b) (2 points) A network classifies images into 3 classes. Why put a softmax on the three outputs instead of a separate sigmoid on each?

Solution

The softmax outputs are positive and add up to 1, so they are one probability distribution over the 3 classes. Separate sigmoids each give a number between 0 and 1, but they need not add up to 1 (all three could be 0.9).

(c) (3 points) A training set has 1,000 records. In one gradient-descent step, every weight in the network is updated once. How many steps does training take in 5 epochs (number of passes through the data) with (i) batch gradient descent, (ii) stochastic gradient descent (one record per step), and (iii) mini-batches of 50 records?

Solution

(i) One step per epoch: 5. (ii) $1{,}000$ steps per epoch: $5{,}000$. (iii) $1{,}000/50 = 20$ steps per epoch: 100.

5. Ensemble methods (10 points)

5.1 (5 points) An SVM can only separate two classes. A clinic wants to classify chest X-rays into 4 classes: normal, pneumonia, COVID and tuberculosis.

(a) With one-vs-rest partitions, how many SVMs do you train, and what labels does each one see?

(b) How is a new X-ray classified?

(c) With one-vs-one partitions, how many SVMs do you train?

Solution

(a) 4 SVMs, one per class. Each sees "this class" against "the other three". (b) Each SVM predicts its side, and every class on the predicted side gets one vote; the class with the most votes wins. (Also accept: pick the class whose SVM gives the largest $f(\boldsymbol{x})$.) (c) One SVM for every pair of classes: $4 \cdot 3/2 = 6$.

5.2 (3 points) An ensemble of 25 classifiers is built, but each base classifier has error rate 0.6. What happens to the error of the majority vote, compared with a single classifier? Why?

Solution

The ensemble is worse than a single classifier: with independent errors, the majority vote is wrong whenever more than half of the classifiers are wrong, and with error 0.6 that happens most of the time (about 0.85). A majority vote only helps when each base classifier is better than random guessing.

5.3 (2 points) Random forests consider only a random subset of the features at each split. Why does this help the ensemble?

Solution

It makes the trees less alike. Without it, every tree would split on the same strong features and make similar mistakes; with random subsets the trees are more diverse, so their errors are less correlated and the vote cancels more of them.